# Mysql & Redis复习

# MySQL

## 执行流程 架构

### 解析器

词法解析

语法解析

### 预处理

检查我的表、列名是不是正确 是不是有权限

### 优化器

选择一种它认为比较好的执行方式 是不是用索引 怎么用都是优化器来决定的

### 执行器

调用存储引擎提供的api

### 存储引擎

MyIsam

InnoDB

### InnoDB储存引擎

内存结构

咕泡出品

Mysql的数据肯定会落盘 数据的可靠性 所以性能会很低很低

bufferpool 提升我的性能 每次操作我只需要跟内存交互

异步刷盘

是不是一定程度上面解决了性能问题 但是也会带来数据一致性问题

会导致内存跟磁盘的数据不一致 假如宕机 内存数据会丢失

**redo** 保证数据的持久性 每次操作还是会跟磁盘交互

redo 内存 跟磁盘 为什么不直接bufferpool每次操作都同步到磁盘

1. redo 顺序IO bufferpool同步 随机IO 性能比bufferpool快
2. page页 bufferpool同步的话会一次同步一个page页（16K） redo一条指令

每次redolog都去同步磁盘 会慢 但是安全性高

可以供用户选择同步方案

0 每秒

1 每次都会同步 默认

2 交给操作系统

**undo** 回滚日志 原子性

**binlog** 主从 同步 数据恢复

mysql的

redo binlog 二阶段提交

2pc

# 索引

知道怎么来的

page页

page存储　链表

书

看书 没有目录的情况　去找一个章节　水浒传　潘金莲　一个章节一个章节去翻

书有目录　潘金莲-----50页

武松打虎--60页

B+

1.不管你什么语句 查询的链路一样的　所以 稳定 但是速度比一定比B树 但是整体上快很多

2.只有最后的一层叶子节点才有数据 所以 page存的数据也就越多 数据越多代表树的高度越矮 越矮 就 查询的次数越少 IO次数越少

聚集索引 非聚集索引

主键ID 聚集索引 是存储数据的索引树

非聚集索引 二级索引　姓名 age

回表

如果我查的数据不需要从聚集索引拿　覆盖索引

select *　一般不用 *　ID

select Id

id查具体的数据

索引下推　二级索引里面有 以一种非索引的形式去实现性能的提升 （减少回表）

age  address

where　and address='长沙'  and age>18  age用到了索引　address用不到

索引下推　　不用回表了

age>18　30条　肯定要回表30条

age>18 30条 address='长沙'　　根据索引查出来以后 会在索引树根据address='长沙' 过滤 2条 只会去回表2条

索引失效条件：

1.范围查询 会使索引列的下一个索引列失效 比如>  <　like XXX%

2.函数运算　　当前索引列失效

3.类型转换 当前索引列失效

4.最左匹配 a b c 假如 b没用到索引 c肯定也用不到

我们一般建立表的时候 一般会把一些大字段单独建一个附表 text varcher(5000)?

# 事务

ACID

并发

带来问题

脏读 读到了其他事务没有提交的数据

不可重复读 一个事务里面2次查询 第二次查询到第一次之后其他事务更改的数据

幻读 一个事务里面2次查询 第二次查询到第一次之后其他事务添加的数据

隔离级别 解决不同的问题

RU

没有解决任何问题 但是性能最高

RC

解决了脏读问题

RR

innodb默认的隔离级别 解决了脏读 不可重复读 幻读

串行化

没有并发 根本就没问题

RR怎么解决　不可重复读　幻读

MVCC 乐观锁

readview

LBCC

悲观锁

事务改数据的时候 其他事务不能改

记录

间隙

加锁一个区间 这个区间内不存在的数据是不能添加的

临键锁

记录+间隙锁

假如有一天慢sql 你会怎么调优?

1.索引　检查有没有用到索引 或者有没有更好的索引方案

2.拆分　把大语句 拆分成跟多的小语句

3.减少回表 尽量不去用* 要什么拿什么

4.一定要有范围限制　一定得分页　减少网络开销

表数据的处理

调整bufferpool大小

分库分表

机器来堆 读写分离 多主 冷热 集群

采用nosql技术 redis　es

# Redis

redis的由来 因为Mysql满足不了需求 内存型数据库

特点:

快

1. 数据基于内存操作 没有磁盘IO
2. kv　底层还有些支持快的数据编码 skiplist 跳表（空间换时间）
3. 命令执行的单线程 （因为在内存操作 速度很快 会出现你线程没切换命令已经执行完）
4. 网络模型 多路复用

咕泡出品

高可用

1.支持持久化

2.内存管理 过期 淘汰机制

3.集群 cluster

**数据类型**

String 缓存 锁 分布式ID

redis为什么能做分布式锁? 单线程

setnx lua+指令

lua能保证多个指令的原子性

不能够并行去抢占一个标记

hash 重入 锁 结构型的数据 统计统计数据 商品 订单数 pv uv

重入锁3个条件:

1.互斥 大key

2.线程信息 小key

3.重入次数 value

list

有序列表 可以重复

set

无序集合

抽奖

集合类的操作

zset

score 排行榜

**内存管理**

过期 每个key都可以设置过期时间的

让我到期的key能够自动删除

1. 被动 对象去访问的时候 判断有没有过期
2. 定期 隔断时间去检查下里面有没有过期

hz 10 100ms会去定期检查一次

redis6.0之后

1.去扫描所有设置了过期时间的key,hash桶的维度 最多扫描20（可配）

2.会去删除这个里面过期的数据 如果过期比例超过10%

3.继续第1.2步

保证 我过期了的数据能够最快的速度从我内存 删除

淘汰

咕泡出品 必屬精品

有可能我数据都没过期 但是内存不足

novication　报错OOM　能读不能写

allkeys-random

volatile-random

volatile-lfu

allkeys-lfu

volatile-lru

allkeys-lru

volatile--ttl

lru

根据时间来淘汰

每个对象都记录了一个时间 会根据系统时间做比较 越久的越容易被淘汰

LFu

时效性问题　次数 、频率

用时间（16bit）+次数（8bit）　根据访问的次数以及配置　越到上面加的几率越低

时间 隔多久没访问的话 去减少响应的次数

**持久化**

内存的数据 ---落盘到 磁盘

RDB

快照文件　保存当时redis里面的所有数据

自动

配饰 save

flushall

shutdown

手动

save

bgsave

快但是数据会丢失

AOF

默认是不开启的 但是开启即默认

同步机制

每秒同步 最多只能有1s丢失

每次同步

操作系统同步

4.0 rdb跟aof的混合模式 默认是开启

aof 变大的时候 会去重写成rdb格式 后续追求的还是指令 恢复的速度慢慢变快

**集群+场景**

**主从**

咕泡出品 必属精品

数据一致性

全量同步

从会发送一个指令给到主 主收到了以后会通过bgsave去生成一个rdb文件然后给到从 从先删除自己的数据，然后通过rdb文件加载数据

主会正常的写数据

增量同步

我先把rdb生成后的数据保存到我的主的内存　当你的从rdb加载好了以后，然后会把增量添加的指令一起给到从。

**cluster**

集成了负载+主从

key1

怎么负载的？　hash算法

虚拟槽 16384

每个主节点都会负责一块虚拟槽的区间

机器1 0-5000

机器2 5001-10000

机器3 10001-16384

key1 % 16384 =0-16383

场景

缓存穿透 击穿 雪崩

咕泡出品 咕泡出品 咕泡出品 

# 调研